

EngageIQ: Smart Engagement Opportunity Scorer

Technical Brief | Big Data (BAX 423) Final Project

Author: Saurav Kanegaonkar | **Student ID:** 924465543

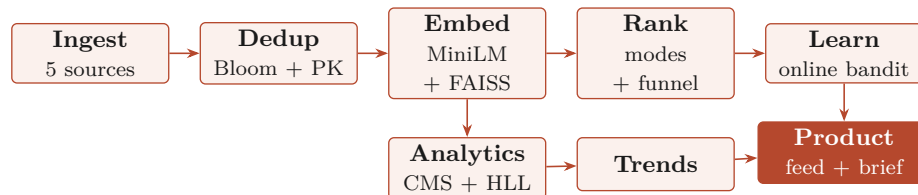
Live application: engageiq-148810228333.us-central1.run.app

Submitted materials:

- `code/` (single-command FastAPI app)
- `data/` (offline 20,995-record SQLite snapshot)
- `brief.pdf` (this technical brief)
- `prompts.md` (the AI prompt log)
- `README.md` (overview and run instructions)

1. System Architecture

EngageIQ reframes the problem as an attention-portfolio recommender: it treats a unit of a professional's time as the item being allocated, and ranks opportunities by expected impact per unit of effort against a weekly budget. The engine is one linear pipeline with a parallel batch-analytics branch, wrapped in a product rather than a notebook.



The pipeline runs ingestion, embedding, ranking, and learning, with a parallel batch-analytics branch, each detailed by capability in Section 2. Two SQLite databases stay separate: a read-only corpus (`engageiq.sqlite`, the snapshot a grader runs with no API keys) and a read-write user-state database (`engage.sqlite`: accounts, profiles, interactions, an event stream, and the learner weights), so the snapshot stays clean while usage persists. The whole app is one FastAPI container on Google Cloud Run with both transformer models baked in, scaling to zero when idle. The offline snapshot holds 20,995 deduplicated, 2026-only records across all 15 required technical domains:

Dev.to	6,607	Reddit	3,768
Bluesky	5,482	GitHub	1,000
Hacker News	4,138	Total	20,995

2. BAX-423 Techniques and Why

All six core capabilities are built from BAX-423 course concepts (the lecture each draws on is cited inline); the algorithmic choices were also benchmarked against a cheaper baseline.

Capability 1 - Multi-source ingestion and dedup

Where. Five source adapters (GitHub, Reddit, Hacker News, Dev.to, Bluesky) poll their REST APIs concurrently with async parallel agents (Lecture 1) and normalize to one canonical Opportunity schema. Incoming records pass a from-scratch Bloom filter (Lecture 2) as a fast probabilistic dedup pre-check, backed by a database primary key, then land in SQLite.

Why. API polling rather than a live Kafka cluster (Lecture 3) keeps the offline snapshot fully reproducible for a grader with no running infrastructure, while reusing the same ingest, idempotent-dedup, and store pattern the streaming lecture teaches. The Bloom pre-check gives that dedup at a fraction of an exact set's memory.

Capability 2 - Embeddings and ANN retrieval

Where. Opportunity text and the user's interest profile are encoded with Sentence-BERT (all-MiniLM-L6-v2, 384-d) and retrieved by exact cosine over an L2-normalized FAISS `IndexFlatIP` (Lecture 5). This is the matching core.

Why. The corpus is short, jargon-dense Reddit topics and GitHub issues where semantic context beats keyword overlap. MiniLM was chosen over `bge-small` for cleaner domain separation. Retrieval precision@10 is 0.89, robust to noisy domain labels.

Capability 3 - Multi-stage ranking

Where. A candidate-generation to re-ranking funnel (Lectures 6 to 7): stratified per-source FAISS recall, hard filters (platform, domain, language), a cross-encoder relevance re-rank, a goal-weighted impact score, MMR for diversity, and a time-budget knapsack that fills the weekly hours. Four engagement modes (Contribute, Discuss, Monitor, Promote) set the signal weights and the action verb.

Why. Dense embeddings alone lose to keyword search on this jargon corpus (Section 3); reading the query and document jointly with a cross-encoder recovers the gap, the funnel reports nDCG@10 (the required metric), and the knapsack turns scores into a realistic weekly plan.

Capability 4 - Adaptive learning

Where. A per-(user, persona) online weight-learner (Lecture 8: a bandit-style Widrow-Hoff update over the nine scoring signals) updates on every engage, skip, or save and persists across reloads.

Why. It personalizes ranking from real feedback, warm-started from the mode defaults and renormalized so the weights stay interpretable. Over 60 simulated rounds, nDCG@5 rose from 0.679 to 0.960 (Section 3).

Capability 5 - Probabilistic sketches and analytics

Where. The batch-analytics layer over the full corpus: a from-scratch Count-Min Sketch (term frequencies) and HyperLogLog (distinct contributors) (Lecture 2), rolled up with PySpark-style window aggregations (Lecture 4), power the Trends tab.

Why. Both sketches merge without rescanning, which is what lets the dashboard roll a persona's domains up on the fly, at a fraction of the memory: 1.16% max error at 9.8x less memory (CMS) and 0.43% at 48.7x smaller (HLL) versus exact counts.

Capability 6 - Recommendation product and dashboard

Where. The Hub presents the ranked feed with a plain-language “Why this” (the scorer’s top signal contributions in words), an LLM-drafted Suggested Action (a concrete post, comment, or pull request) built with structured JSON prompting (Lecture 1), plus the Trends tab and the exportable brief.

Why. The engine is wrapped in a product so a non-data-scientist can act on it: “Why this” makes the ranking transparent, and the Suggested Action turns a match into a concrete next step rather than just a link.

3. Matching and Ranking Benchmarks

The Stage-2 re-ranker was benchmarked with an LLM-as-judge harness (relevance 0 to 3, pooled and cached for reproducibility), reporting nDCG@10 and P@10.

Stage-2 matching strategy	nDCG@10
Dense (MiniLM cosine, alone)	0.63
TF-IDF (keyword)	0.77
Hybrid (dense + keyword)	0.70
Cross-encoder re-rank (chosen)	0.76

The honest finding: dense embeddings alone (0.63) underperform plain TF-IDF (0.77) on this short, jargon-heavy corpus. The design treats this as the reason for the cross-encoder Stage-2 (0.76), which matches keyword quality. The full assembled pipeline (recall, filters, cross-encoder, impact, MMR, knapsack) reaches a mean nDCG@10 of 0.711 across the four test personas. Adaptive learning was validated over 60 feedback rounds (nDCG@5 0.679 to 0.960). A 53-check automated suite covers the sketches, analytics, and the trends and brief endpoints; all pass.

4. Test Persona Results

Each of the four provided personas was evaluated against its pass criteria and each of the six core capabilities. All twelve persona pass-criteria pass; the capability coverage is below.

Core capability	Sofia	David	Lina	Raj
1. Ingestion + dedup + storage	Pass	Pass	Pass	Pass
2. Embedding + similarity retrieval	Pass	Pass	Pass	Pass
3. Scoring + multi-stage ranking	Pass	Pass	Pass	Pass
4. Adaptive learning from feedback	Pass	Pass	Pass	Pass
5. Batch analytics + trend detection	Pass	Pass	Pass	Pass
6. Dashboard + Why-this + brief	Pass	Pass	Pass	Pass

Each persona’s pass criteria, all checked on the top-10 feed. **Sofia (Contribute):** at least 3 GitHub repos, zero C++/Rust, ML focus (6 of 10). **David (Discuss):** infra focus (6/10), discussion-type items (6/10), at least one standout repo (high activity, few contributors). **Lina (Monitor):** a top-10 fresher than the corpus, velocity emphasised, at least two sources. **Raj (Promote):** developer-tools relevance (6/10), discussion threads rather than bare links (6/10), and no drift into general programming (at most 2 of 10).

5. Limitations

Snapshot recency bias. The offline scrape skews to recent weeks (a collection artifact), so a raw volume-over-time line would mislead. Trend signals therefore use each domain's share of weekly activity on complete weeks only, which is robust to the bias, and the limitation is stated in-product.

Domain-label precision. Labels began as keyword matches (about 62% precision). A multi-label re-classification with an independent verification gate lifted this to 78%, but several niche domains (for example Python data engineering, AI research) remain weaker, so a small fraction of items carry an imperfect domain tag.

GitHub supply. GitHub is the smallest source (1,000 records), so a Contribute-heavy persona can see fewer GitHub items than its plan requests even though the matching is correct; the candidates exist in the corpus but do not always rank into the top slice. This is a data-supply limit, not a ranking bug.

Hosting trade-offs. The free Cloud Run instance has an ephemeral filesystem, so runtime user accounts and learned weights reset on a redeploy or a cold start; the read-only corpus and all ranking are unaffected. Durable accounts would need a managed database, which was out of scope for a free single-instance deployment.